学习与感悟

NCUSCC 2026 秋季招新考核



2026 年 9 月中旬 - 10 月中旬

一、 跨专业起步：从 DIY 的念头到敲下第一行命令

1.1 报名契机、DIY 初心与对命令行的最初印象



我是材料科学与工程专业的本科生。因高考分数的原因我来到了材料专业，但自己从小就觉得计算机很酷，后来又对人工智能产生了兴趣。平时在用 GitHub 上许多开发者分享的程序和作品时，我常想：网上的现成资源虽然多，但总会有一些新需求找不到现成方案；如果我也能像 DIY 一样自己做些简单的程序或插件，或者借助 AI 把它们做出来，既能解决自己的需求，也能帮到和以前的我一样不懂这些的人。后来有同学向我介绍超算俱乐部（NCUSCC），能在俱乐部里学习自己真正喜欢的东西这一点打动了我，于是决定报名试试。

在动手准备这次考核之前，我已经借助 AI 梳理过大致的学习流程，但真正起步时还是充满了生疏感。平时我用电脑基本都在 Windows 图形界面下操作，回看 9 月中旬的记录，我在 2026 年 9 月 17 日问过“vs code要不要学”，9 月 25 日又问“linux要学什么”，还追问过 Linux 在实际开发里有什么作用和优势、平时在 Windows 下能不能直接做。那时候 Linux、Ubuntu、WSL、Git、GitHub、VS Code 这些名字对我来说还混在一起。不过我对命令行一直有种特别的好感：以前觉得命令行只有高手才会用，它用简洁的指令代替了鼠标点击，给我的感觉就像把一个工具的外壳拆掉了，只保留里面最核心的组件一样。

1.2 从 Minecraft 指令排错到 Linux 终端的初次试错

在接触 Linux 之前，我离“敲指令”最近的一次经历是以前玩 Minecraft时给三四个服务器做指令功能。当时我常参考网上的作品和思路，学习别人写的技能或抽奖动画后修改参数融入服务器，并非全部原创；自己比较有印象的一次原创尝试，是做了一个把石头“锻造”成矿物的动画装置，因为当时还不会做炫酷的粒子特效，基本也没收到什么反馈。那时候 AI 还不发达，遇到指令不生效全靠自己排错：我会先看命令方块的输出判断大概哪里出了问题；如果看不出来，就把长指令链从前往后拆开（比如只留第一句，对比预期和实际结果），有时也反过来从后往前拆；实在找不到原因，就新建一个空白存档排除干扰，一步步试。



图 1 拆解长链与新建空白环境排查问题的过程示意。

因为有过这段自己拆指令、建新档排错的经历，我起初有些乐观，以为花大约一个月就能上手 Linux。我主要利用国庆假期、按事情的紧急程度安排时间，在 Windows 11 电脑上配好 WSL2 Ubuntu 环境，跟着 B 站《Linux 基础》视频学习——这门课比较基础简单，采用“先解释命令、再给演示”的讲法，很适合我。看到第三章时我主动停下来要求针对第二章布置练习，并在本地建了 chapter2-practice 目录实操。

真正敲起命令后，我发现以前在 Minecraft 里拆指令链的经验并不能直接顺畅地迁移过来：Linux 的命令本身很陌生，面对的又是全英文报错，加上以前基本没有实际操作经验，操作复杂度一上来就容易卡壳。比如 10 月 1 日练习用户与权限时，我先敲 useradd -g test 收到用法提示，补上用户名敲 useradd genshin -g test 后执行 su - genshin，又遇到无法进入 /home/genshin 的报错；把报错贴出来追问“讲讲这几个参数呢”并试过 userdel genshin 后，才分清位置参数、-g 指定已有主组、创建家目录选项与同名用户组的关系。同一天配置免密 sudo 执行 chmod 440 /etc/sudoers.d/nopasswd 时，我还因为终端没有任何文字回显而专门问“这个没有输出”，直到 10 月 6 日练习 chmod 640 并用 ls -l 看到 -rw-r-----，才慢慢习惯通过查状态来确认命令有没有生效。

在 10 月初这批练习里，我确实有过自己看懂报错、把命令改对时的高兴，只是现在已经记不清具体对应哪一次操作了。从留下的终端记录看，当时的练习大多是在磕磕绊绊中小步推进：比如 10 月 2 日凌晨练习 cp 遇到源文件缺失时，先用 grep 生成文件再复制、修正 wc 路径算出 3 行结果；10 月 6 日下午做目录练习漏看了“单条命令创建嵌套目录”的约束，就先分步建出目录、修正切换路径完成 ../backup 备份；练进程管理时曾把输出里的 PID 数字直接当命令敲进去搞出 333 command not found，到 10 月 8 日也在分步提示下做完了 top -p 494 和 kill 后的复查。

二、 寻找合适课程与厘清环境边界：独自摸索中的曲折

2.1 找课的曲折、独自求助 AI 与理清工具角色

回想准备考核的前期阶段，虽然大致的学习流程之前已经借助 AI 梳理过，但我当时遇到的一个很大困难是**找不到适合自己基础的课程**，而且身边也没有能直接请教的人——认识的同学不会这些，不认识的学长或群友我又不好意思去问，既怕问题太基础打扰别人，也担心自己连问题都说不清楚，所以遇到卡点基本只能向 AI 求助。

这种“找不对课又没人可问”的困扰在初学 Git 时最明显。10 月 8 日下午我刚开始找 Git 命令行教程时，点开过一个讲得很快的视频，默认观众已经有开发基础；当时看到弹幕里有人刷“没有基础你学什么 Git”，我心里挺不是滋味：一方面觉得那条弹幕说话不太客气、这个视频确实不适合自己并果断换了视频，另一方面也确实有一点怀疑自己，担心是不是学习方向选错了。换掉视频后，我在当天下午向 AI 追问了“git是linux文件吗，怎么命令跟Linux这么像”，10 月 12 日又补问了“SSH远程连接是什么”。



把这些问题一点点问清楚后，我才理顺了工具之间的层次：平时面对的黑色终端窗口只是 Shell 交互界面，在里面敲 ls、cp、ps 是调用 Linux 系统自带的工具，敲 git init、git status 是调用安装在系统里的 Git 版本控制程序，而 SSH 则是连接远程机器的通道。

图 2 在不同教程间摸索筛选适合零基础课程的过程。



图 3 操作系统、Shell 终端、Git 与代码编辑器的职责分工。

2.2 本机 Windows、本地 WSL 与网页考核容器的隔离

理清工具角色之后，紧接着遇到的是不同环境之间的差异。跟视频课练习时，教程里演示的是传统虚拟机，而我自己电脑上装的是 Windows 11 下的 WSL2 Ubuntu，两边在默认 root 密码、重启方式等细节上不完全相同（10 月 1 日我就专门问过 WSL 默认 root 密码和怎么重启 WSL），遇到和教程不一致的现象时经常要贴出报错求助。

到了正式做考核题时，又多了一层招新平台网页里的题目容器。我把提醒写在了闯关笔记开头：题目网页里的 /workspace 是考核容器内的独立路径，既不是 Windows 本机目录，也不是本地 WSL 里的同名文件夹，而且不同关卡的容器各自拥有独立的文件、账号和 Git 仓库基线。动命令前先看清当前主机身份、当前用户和工作目录，能省去很多跨环境操作的折腾。



图 4 本机 Windows、本地 WSL2 Ubuntu 与考核网页 /workspace 容器的环境隔离。

2.3 立足当前基础，将进阶重心调整为 C/C++ 赛道

按照 10 月 8 日转发的招新通知，完成基础题后需要在 HPC、CUDA、C++ 三个进阶方向中三选一。起初我考虑过 HPC 方向，但在 10 月 12 日结合自己的零基础现状、当时的剩余时间和可用工具条件综合考虑后，我把重心调整为优先完成 C++ 方向。

按这一顺序，10 月 12 日至 13 日完成了 Linux 7 关、Git 6 关与 C/C++ 4 道必修题（共 17 项）。到了 10 月 15 日，抱着“想至少写完或学完吧”的想法（过程中确实也有过想停下来、不继续做的时候），我又顺着平台任务地图把 C/C++ 的 4 个可选方向也推进完了——其中前 20 项（17 项必修及“读数变化观察”“馆藏目录卡”“借阅台账”3 项可选）通过了平台自动检查，第 21 项“四线程快速排序观察”完成了实验与 112 组不同测试数据验证，并将观察报告 report.md 上传提交至平台（平台校验文件格式后标记“已收件”，报告内容待人工审阅）。



图 5 结合当前基础选择 C/C++ 赛道并循序推进的考核路径。

三、 闯关过程中的预期落差与具体认识变化

如果把整个闯关过程串起来，我的实际推进方式经历了几次变化：刚看完入门视频时觉得自己学得还不错 → 在 Linux 第 1 题里先判断需要命令做什么、向 AI 问清可用命令后**自己动手去终端试通**，觉得现学现用还能推进 → **到第 4 题突然卡住**、尝试简单命令后仍不知道怎么组织完整步骤，求助 AI 的方式也从“让它一步步教我”改为“**先让它完成、再对照完整做法回头拆开理解**” → 转到 Git 时**重新接上之前学过的基础**，再到 C/C++ 面对陌生代码与概念通过事后追问一点点补课。



图 6 从以为自己已经学会到看见庞大未知知识冰山的落差。

3.1 Linux 第 1～3 关：第一次自己动手试通的高兴，与现学现用下的推进

刚看完 B 站那几章 Linux 入门视频时，我本来觉得自己学得还不错。可第一次打开招新平台的 Linux 第 1 题时，我还是直接愣住了——题面里涉及的操作和命令看着很陌生，加上以前基本没有实际操作经验，一时反应不过来。定下心后，我的做法是先判断每一步需要命令完成什么，再问 AI 有什么命令可以实现，觉得符合题意后自己到终端里去尝试（当时还不太能提前判断全部结果）。



顺着这个办法，我把 /workspace/input.bin 复制到程序 /workspace/exam 固定读取的 /workspace/input，用命令替换传入 /workspace/cabinet/key.txt 里的密钥，再把固定写出的 /workspace/output 存成 /workspace/flag，自己动手试通了第 1 题。当网页平台弹出通过窗口的那一刻，我心里非常高兴，稍微缓了一下就接着去做第 2 题了。当时虽然觉得入门视频没有覆盖所有命令，但现学现用还能往前推进；不过我也已经感觉到，自己基础不足依然是个很大的障碍。

接着往下做第 2 关和第 3 关时，我结合早期本地练习里的一次教训——当时大概是因为看错题，重复执行了一次追加操作让 6 行文件变成了 12 行，虽然最后汇总表面上看似正常，但其实输入已经翻倍且当时没有重建清理——开始留意核对固定路径、中间文件和实际字节。比如第 2 关连续处理 A、B 两份输入时，跑完 A 要先把 /workspace/output 转存为 A.out，再用覆盖方式把 B 拷入 /workspace/input（不能沿用第 1 关防覆盖的 cp -n），最后按 A → B 顺序拼接并用 cmp 比对；第 3 关原文件 REPAIR THIS SIGNAL\n 算上末尾换行符 LF（0a）共 19 字节，为凑齐 4 字节密钥 NOVA 的整数倍，用 sed 在最后一行 LF 前补 1 个空格变成 20 字节，再通过 wc -c、od -An -tx1 和逆向删空格后的 cmp 确认只改了这 1 个字节。

3.2 Linux 第 4～7 关：第 4 题的“当头一棒”、求助 AI 方式的转变与后续压力



真正打破我“现学现用就能一路做下去”这种想法的，是 Linux 第 4 题。到了第 4 题，我感觉难度突然上升，像挨了“当头一棒”。起初我还想靠自己做，反复读题目要求、查里面的陌生词，也在终端里试过一些简单命令，但始终组织不出完整的操作步骤。当时最让我受挫的，并不是写了很久的脚本一直在报错，而是根本不知道该从哪里开始，没办法像以前排查 Minecraft 指令链那样一节一节拆开试。

也是从这里开始，我求助 AI 的方式发生了明显变化。最开始我要求 AI 一步步教我，自己盯着它的操作看：单个命令拆开看还能勉强理解，可一旦把变量传递、条件判断、退出码处理等多个步骤连在一起，前面的内容我还没消化完，后面又冒出了新东西。边看边跟着做已经很难往下推进，所以后来我改成了**先让 AI 把题目完成，再对照完整做法回头拆开理解，遇到不懂的地方继续追问**。看到 AI 处理第 4 题时也反复尝试了好几轮，我更觉得这道题确实超出了自己当时的基础（后面的第 5 题印象里同样很难，只是具体记忆没有第 4 题这么深了）。

到了后续这几关，因为我已经习惯了 AI 的能力，看到它把题目跑通时我并不会觉得特别高兴；反而是看着题目越来越难、冒出来的陌生内容越来越多，心里反而更慌了——既担心后面的内容连讲解都听不懂，也担心自己之后无法复现、会影响考核结果，当时甚至有过“感觉白学了”的强烈冲击。

这几关里的具体机制，都是我在 AI 辅助完成后，回过头看完整做法和追问才逐步了解到的：比如第 4 关 route.sh 里普通变量要用 export ACCESS_TOKEN 才能传给子进程，task-probe 返回的 11 或 23 要立即通过 $? 捕获并映射成退出码 75 或 65；第 5 关管理后台进程时不能只看 worker.pid 里的数字，要去 /proc/PID/ 下核对进程状态、命令行、工作目录与环境变量，并且子 Shell 启动后台任务前要用 exec 9>&- 关掉继承的锁文件描述符；第 6 关和第 7 关则是通过复盘了解到，容器里的 root 读 000 权限文件也会报 Permission denied，验证 visitor 拒读前要先确认账号本身能切换，以及编译失败时要先看 build.log 解决缺 gcc 和冲突包 ncuscc-build-guard，再修补 /workspace/hello.c 里 printf 漏掉的分号。



图 7 从“只看表面报错”转向“依靠日志、进程接口与调试工具获取证据”的排错闭环。

3.3 Git 6 关：重新接上熟悉基础的缓和感，与事后梳理的恢复及分支机制

从 Linux 后几关转到 Git 模块时，整体感觉没有 Linux 后几关那么难。因为 Git 前面学过一点基础，重新遇到熟悉的命令让我觉得有些内容终于能接得上了，不再像前面那样每一步都完全陌生：比如第 1 关 git init -b main 后停在还没有提交的“未诞生分支”（索引为空、notes.txt 为 ?? 未跟踪），第 2 关只改工作区文件而不暂存（git status --short 显示 M notes.txt，git diff --cached 为空），第 3 关用 git add 和 git commit 留下第一个版本。

后面几关涉及到历史恢复与分支合并时，我是通过回顾之前学过的内容并结合 AI 辅助往前推进的，并没有某一个印象特别具体的 Git 卡点。作为做完之后整理出的知识总结，第 4 关（Git 恢复）和第 6 关（Git 综合）主要帮我区分了不同恢复方式的作用条件：在第 4 关中，不带参数执行 git restore notes.txt 默认是从当前暂存区（c1 的 version two）恢复工作区，若加 --staged 则会改动暂存区；只有先用 git rev-parse HEAD^ 解析出真实提交 ID，再显式执行带 --source="$recovery_source" --worktree 的 git restore，才能在本题中只把工作区恢复成 version one，同时保持暂存区和 HEAD 留在 c1 不变。而在第 6 关中，main 通过 git merge --ff-only 快进合入 trial 的修改后，用 git revert --no-edit 创建反向提交，使文件树（HEAD^{tree}）和 notes.txt 回到基线，同时把修改与撤销两次提交都保留在历史记录里。



图 8 git revert 实现“文件内容还原至基线、修改与撤销历史同时留存”的版本控制机制。

3.4 C/C++ 必修与可选题：从先找眼熟结构到通过事后讲解认识调试与内存问题



C/C++ 的 4 道必修题和 4 个可选方向对我来说是最吃力的部分。面对这些题目的代码时，我会先去找输入输出、循环这些眼熟的结构，但各个部分之间具体是怎么连起来的仍然不清楚，更看不出错误可能出在哪里；像 GDB 调试、内存布局、四线程并发这些内容我原本都不会，很多工具和名词以前也没听过。

即便当时暂时理解不透，对这个方向的兴趣和“想把这件事完整做完”的念头还是让我继续往下走，希望先把这次考核完整走一遍，之后再慢慢补回来。

因此，这一模块里的具体解法并不是我做题当下自己诊断、设计和实现的，而是在 AI 完成代码与测试后，我通过回看讲解和报告才逐步认识到下面这几类问题：

一是**首项与极值边界**。看讲解后我了解到，必修第 1 关用第一个读入的真实元素初始化最小值和最大值，可以避免初值设为 0 在全正或全负数下出错；可选“读数变化观察”（cpp-optional-debug）对单个数 4 输出错误的 increases=1，则是通过 -g -O0 编译并在 main.cpp:19 设 GDB 断点，看到 i=0, previous=0, value=4 把初值 0 误当成了前项，加上 if (i > 0) 限定从第二项开始比较才修好。

二是**数组越界与共享缓冲区**。必修第 3 关通过 -fsanitize=undefined 复现了下标 8 越界，改成容量 8 的环形窗口后，满窗口时先从 sum 减去 window[next] 旧值再覆盖写入，并按回绕位置

区分最旧值与最新值。可选“馆藏目录卡”（c-optional-layout）原模板用指针指向同一个 char shared[9]，只复制地址导致输出全变成最后一条 CD，改成结构体内置独立的 char code[9] 并用 strlen 配合严格大于 > 更新最长下标才解决；可选“借阅台账”（cpp-optional-raii）则用 std::map 管理库存，注意区分不带数量参数的 return code，借出与归还前先用 find 检查，任一步失败立即只输出 rejected。

三是**编译链接与整数宽度**。必修第 4 关报错 undefined reference to print_report(int, long long, int, int)，回看分析才知道是 CMakeLists.txt 漏了 format.cpp，且 format.cpp 定义的参数类型顺序与头文件声明不一致。同时，必修第 4 关总和可达 10^13，必须用 long long 防 32 位溢出；而另外三道可选题在给定约束下上界未超出 32 位有符号 int，使用 long long 是为了遵守题面“至少使用 64 位整数”的要求。

四是**逻辑 CPU 亲和性与小规模并发耗时观察**。在可选“四线程快速排序观察”中，主线程通过两层三路划分把 16 个固定整数拆成至多 4 个互不重叠区间，固定创建 4 个 std::thread 工作线程排序并 join。环境检查中 os.sched_getaffinity(0) 显示该进程允许运行的逻辑 CPU 数为 **1**（cpu.max 不可读，不能据此直接断言宿主机只有一个物理核心，也不能当作容器 CPU 配额的完整测量）；20 次独立运行计时最小为 1956 微秒、中位数为 2107.5 微秒、最大为 2799 微秒。在仅 16 个元素的小规模任务且允许运行的逻辑 CPU 数为 1 的条件下，线程创建、调度和汇合等额外开销可能影响总耗时，但当前实验未测定这些开销的具体占比，也没有串行基线，不足以判断相对串行实现的加速效果。

四、 事后追问方式、当前掌握边界与后续打算

4.1 怎么让 AI 讲懂：在反复追问中按步骤拆解消化

在全部题目做完之后回看讲解的过程中，我也更清楚自己能听懂什么样的解释。面对陌生的脚本或代码，我不适合一次被塞进整段复杂实现或一大堆概念，**更适合先知道“这一步到底要做什么”，再听“命令或代码里各个部分的含义”，最后看“实际演示与运行结果”**。而且当多个陌生概念一起出现时，并没有哪个瞬间让我一下全都突然听懂了——往往 AI 解释一次我还记不住，需要隔一会儿再回头追问同一个问题。同时，在整理报告和配图时，我也会用自己刚弄明白的逻辑去检查 AI 生成的解释图——比如指出 Git 分支图里箭头指向不明确、文件名称遮挡图片等问题并让它修正。



经过这一轮回看和追问，原先许多连名字都没听过的工具和名词，我现在正在逐步理解，并不是交完题还什么都没学会；但我也很清楚，**完成提交、能跟着解释看懂，绝不等于自己已经独立掌握或完成了独立复现**，目前也还有一些细节没完全消化：

Linux 基础与系统管理（L1–L7）

我自己动手或查资料完成的部分

本地 WSL 基础练习（mkdir/cp/grep/wc/useradd/chmod/ps/top/kill）；第 1 关先判断需求、向 AI 询问可用命令并判断是否合题意后自己动手到终端试通，继续推进第 2、3 关基础操作

闯关时由 AI 完成、我通过事后讲解与追问逐步理解的部分

第 4～7 关的整体目标与排错脉络（事后听懂了为什么需要 export、退出码映射、去 /proc 核对进程身份、640 权限正负向验证及先装编译器再修语法）

目前仍在消化、尚不能独立写出的部分

L4～L7 里的复杂 Shell 脚本写法（如 flock 锁描述符关闭 exec 9>&-、/proc 空字节分隔读取、完整的参数匹配与脚本结构），目前还写不出来

Git 版本控制（G1–G6）

我自己动手或查资料完成的部分

前几道学过基础的关卡（初始化 main 分支、查看工作区与暂存区状态、基础 add 与 commit），以及核对配图中的分支与父提交箭头关系

闯关时由 AI 完成、我通过事后讲解与追问逐步理解的部分

后面几关中带 --source=… --worktree 的 git restore 仅恢复工作区、同基线分叉 --no-ff 双亲合并，以及 --ff-only 快进后用 revert 反向提交的作用差别

目前仍在消化、尚不能独立写出的部分

闯关时用于验题的底层只读命令组合（如 git ls-files --stage、HEAD^{tree} 比对），以及不看提示独立完成复杂分支恢复的操作熟练度

C/C++ 必修与可选方向（C1–C4、O1–O4）

我自己动手或查资料完成的部分

初看代码时能先找出输入输出、循环等眼熟的基础语法结构

闯关时由 AI 完成、我通过事后讲解与追问逐步理解的部分

首值初始化极值、首项 i > 0 边界、结构体独立字符数组替代共享指针的基本原因，以及 C3 容量 8 环形窗口下标回绕、O3 std::map 台账查找校验、C4 多文件 CMake 与函数签名对齐、O4 四线程快排区间划分与逻辑 CPU 亲和性观察的整体逻辑

目前仍在消化、尚不能独立写出的部分

脱离 AI 独立编写和调试 C/C++ 程序、GDB 与 ASan/UBSan 的实际操作，以及四线程快排三路划分和 std::thread 并发控制的代码实现

4.2 真实停留的位置：接下来继续学和动手练

回想这一路，遗忘、觉得难和反复其实一直都在发生。早期做完前几道 Linux 练习题隔了一段再去练时，我就直接反馈过“第五题忘记了”“后面的忘光了”，只能回头重新复习；10 月 6 日晚上看一段很长的 ps/top 进程讲解时，我也因为一次塞进来的概念太多而说“好难！要不把不重要的简化掉吧”，当时觉得 Linux 太多太难，还动过念头想先去学其他内容。后来之所以又回来对助手说“继续学吧，把第十题告诉我”并继续交操作记录，一方面是去看了眼别的内容发现好像更难，另一方面也是觉得既然开了头，不把这件事做完心里总觉得不舒服。



把全部题目做完交上去之后，我心里既有松了一口气的感觉，也有觉得疲惫、甚至没有什么特别强烈感受的时候；并没有那种很强的“我已经会了”的成就感，因为心里很清楚还留着许多需要理解的内容。不过回头再看，虽然中途遇到难关时确实有过“感觉白学了”的冲击，但后来回顾 Git 和基础命令时也能看出来，之前的学习并不是完全没用，只是当时的基础还不足以应对复杂的综合任务。

接下来我的计划很实在：顺着已经梳理好的学习流程继续往下学、亲手敲命令和写代码练习，同时让 AI 按“先讲清这一步要做什么、再拆解命令各部分含义、最后看演示结果”的方式，帮我把这次考核里还没完全看懂的内容一点点补回来。

同时，我也真心希望有机会加入超算俱乐部：在独自摸索的这段时间里，我特别希望能有学长和同伴给出指导、在卡住的时候有人可以请教、能一起做些程序或小项目。如果以后我把基础打牢了，我也很想做出能真正帮到初学者的工具或整理出入门经验，让和当初的我一样从零起步的同学少走一点弯路。



图 9 拆分小步目标、逐项追问消化与持续补齐基础的学习过程。